De La Salle University
College of Computer Studies

SYSADMN Laboratory Manual

Name (LN, FN):	Quizon, Marc Lesley E.	Date:	**Jan 27 2026**
Section:	S18	Grade:	

5.0 Unix / Linux Command Line Interface (CLI) Basics – Advanced File Commands

The Unix / Linux system has advanced file commands for file management. Advanced file commands include checking of disk usage and free space, creation of hard and soft links, and setting of permissions. Hard links are additional names for a file. A file can have multiple hard links. Soft link on the other hand is a pathname to a target, which is also called symbolic link. File permissions allow setting of read, write or executable attributes of a file for a specific user or set of users. The objectives of this section are:

- Be able to check disk usage and free disk space
- Be able to create and use hard link and soft link for files
- Be able to set file permissions for files

5.1. Disk Usage and Disk Free

The Unix environment allows the user to check his disk usage and the available disk space in the file system. The commands are "`du`" and "`df`" respectively.

The "`du`" command allows a user to his/her disk usage including usage in subdirectories. By default, the disk usage is presented in 1024 bytes per block. If a user sees that his/her disk usage is 108 blocks, this means that it is really 110592 bytes total (108*1024 bytes per block).

The "`df`" command shows the disk statistics of a file system. Typing "`df`" on the command prompt will produce an output on the screen as shown in Fig. 5.1:

```
Filesystem    1k-blocks     Used   Available   Use%  Mounted on
/dev/sda2       3826584   368332     3263868    11%  /
/dev/sda1         46636     6064       38164    14%  /boot
None              63352        0       63352     0%  /dev/shm
```

Fig. 5.1 Sample output of the df command

Looking at the sample output above, the *Filesystem* column shows the partition/s on the server. The next column, *1k-blocks*, shows the capacity (total size) of the partition. The *Used* column shows how much is already used on the partition, while *Available* shows how much can still be used. The *Use%* shows how much percentage is used on the partition. The *Mounted on* column shows what is the directory name of the partition. **Take note that 1k-blocks is equivalent to 1024 bytes**.

In the example above, partition `/dev/sda2` shows that it has a capacity of around 3.6Gbyte. On the same partition, around 359Mbyte is already used with still 3.1Gbyte available. Data on the partition occupies only 11% of the disk space. This partition is mounted as the root (the meaning of mounted will be discussed later in the course). **Before starting the exercise, delete all files in your home directory by entering the command "`rm *`". Do not delete directories.**

Step 1 At the command prompt, type the command "`du`" and press enter. The command prints the size of the files in 1K block size. Observe the numbers. The very last number is the total blocks all your files are using.

1a. How many directories do you have? How many **blocks** are you using in your directory? *Note that a block is 1024 bytes.*

I have a total of 11 directories and I am using 72 blocks in the directory.

```
lbysyads18user20@SYSSVR1:~$ du
4       ./.cache
4       ./.config/procps
8       ./.config
4       ./coffees/beans/robusta
8       ./coffees/beans
12      ./coffees
4       ./dir2
4       ./dir1
4       ./.local/share/nano
8       ./.local/share
12      ./.local
72      .
lbysyads18user20@SYSSVR1:~$
```

1b. At the command prompt, type the command "`du -h`". The option/switch used in the command shows a human readable form. How many kilobytes are you using in your home directory?

```
lbysyads18user20@SYSSVR1:~$ du -h
4.0K    ./.cache
4.0K    ./.config/procps
8.0K    ./.config
4.0K    ./coffees/beans/robusta
8.0K    ./coffees/beans
12K     ./coffees
4.0K    ./dir2
4.0K    ./dir1
4.0K    ./.local/share/nano
8.0K    ./.local/share
12K     ./.local
72K     .
```

I am using 72 kilobytes in the home directory

1c. At the command prompt, type the command "`du -d 1`". How is the output different from the "`du`" command with no options?

The key difference I saw less directories were shown, I only see 6 lines. I believe this command only shows the parent directories. Hence, the decrease in lines.

Step 2 At the command prompt, type the command "`cp /home/files/rfc1034.pdf rfc1034.pdf`" and press enter.

2a. Type the command "`du -ch`" on the command prompt and press enter. Did the total number of blocks used increase?

Yes the total number of blocks increased

2b. How many **bytes** are you using now in your home directory?

204 000 bytes. I calculated this by taking 204K and multiplying it with 1000.

Step 3 At the command prompt, type the command "`df`".

3a. How many bytes are there per block if this command is used?

1024 bytes per block

3b. How many bytes is the capacity of `/dev/mapper/pve-vm--100--disk--0`?

Capacity – 102626232 * 1024 = 105089261568 bytes

3c. How many bytes are used on `/dev/mapper/pve-vm--100--disk--0`?

Capacity – 2782952 * 1024 = 2849742848 bytes

3d. How many bytes are available on `/dev/mapper/pve-vm--100--disk--0`?

Available – 94584016 / 1024 = 96854032384 bytes

3e. What is the percentage of data used in `/dev/mapper/pve-vm--100--disk--0`?

> 3% of data is used. This information was taken from the Use% column

3f. At the command prompt, type the command "`df -m`". What is the size of each block in this command with option?

> -m is an option that displays the data using 1M-blocks. The size of each block is 1 mb (1,048,576 bytes)

3g. At the command prompt, type the command "`df -h`". What is the difference of this output from the output of the "`df`" only (no options) command?

> The command 'df -h' displays the data using the greatest storage unit . -h stands for human-readable. This is easier to read compared to the 'df' command. The 'df' command does not display the data with any unit, it displays the number of 1K-blocks

3h. At the command prompt, type the command "`df -h rfc1034.pdf`". What is the output? Which disk does this file use?

```
lbysyads18user20@SYSSVR1:~$ df -h rfc1034.pdf
Filesystem                          Size  Used Avail Use% Mounted on
/dev/mapper/pve-vm--12400--disk--0   98G  2.7G   91G   3% /
lbysyads18user20@SYSSVR1:~$
```

The disk used is /dev/mapper/pve-vm--12400--disk--0

5.2. Hard Links and Soft Links

A *file link* is a directory entry for a file. Each file has a file name entry with a corresponding information entry that allows access to the file. A file can have two types of links: hard link and soft link (also sometimes called symbolic link). A hard link creates **a new file name entry for a file** but **does not create another copy** of the file (this is synonymous to having two pointers point to a file) while a symbolic link creates **a new file entry pointing to the original file entry**. Most of the time, symbolic links are more useful since you can use it to create "shortcuts" to a directory.

Step 4 At the command prompt, type the command "`du -ch`".

4a. What is your total disk usage?

> My total usage is 204K

4b. At the command prompt, type the command "`cp /home/files/rfc2186.txt rfc2186.txt`" then "`cp rfc2186.txt rfc_2186.txt`". This will create a copy of the text file "`rfc2186.txt`". Verify if the file was created using the "`ls -l`" command. Was the file created?

```
lbysyads18user20@SYSSVR1:~$ cp /home/files/rfc2186.txt rfc2186.txt
lbysyads18user20@SYSSVR1:~$ cp fc2186.txt rfc2186.txt
cp: cannot stat 'fc2186.txt': No such file or directory
lbysyads18user20@SYSSVR1:~$ cp rfc2186.txt rfc2186.txt
cp: 'rfc2186.txt' and 'rfc2186.txt' are the same file
lbysyads18user20@SYSSVR1:~$ ls -l
total 164
drwxr-xr-x 3 lbysyads18user20 students   4096 Jan 13 16:45 coffees
drwxr-xr-x 2 lbysyads18user20 students   4096 Jan 20 16:25 dir1
drwxr-xr-x 2 lbysyads18user20 students   4096 Jan 20 16:25 dir2
-rw-r--r-- 1 lbysyads18user20 students 131676 Jan 27 20:38 rfc1034.pdf
-rw-r--r-- 1 lbysyads18user20 students  18808 Jan 27 21:25 rfc2186.txt
```

Yes the file was created

4c. At the command prompt, type the command "`du -ch`". What is your total disk usage? Did your disk usage increase? Why or why not?

> My disk usage is now 244K, it has increased. It increased because we copied rfc2186.txt into our home directory and made a copy (rfc_2186.txt)

4d. Add the text "Hello World!" on the first line of the "`rfc2186.txt`" file using any text editor and save the changes. View the contents of the file using the command "`more rfc2186.txt`" (to exit type "`q`"). Also view the contents of the "`rfc_2186.txt`" file. Modify the "`rfc_2186.txt`" file this time, save the changes, and view both file contents. Do both files have the same exact content? Why or why not?

> The two files do not have the same content. This is because the command "cp" creates a copy of a file and they become independent of each other. Changes to the copy or original will not affect one another.

4e. Delete the file "`rfc_2186.txt`" and check your disk usage using the command "`du -ch`". What is your disk usage now?

> My disk usage is now 224K.

4f. At the command prompt, type the command "`ln rfc2186.txt rfc_2186.txt`". This command creates a hard link for the file "`rfc2186.tx`t" text file. Verify if the hard link was created using the command "`ls -l`". Was the hard link created?

> Yes it was created.

4g. At the command prompt, type the command "`du -ch`". What is your total disk usage? Did your disk usage increase? Why or why not?

> It did not change, it is still 224K. This is because hard links do not create a copy of a file. Hence, disk usage does not change

4h. View the contents file "`rfc2186.txt`" by typing the command "`more rfc2186`.txt", to exit type "`q`". Also view the contents of the file "`rfc_2186.txt`" using the command "`more rfc_2186.txt`", to exit type "`q`". Were you able to view both files?

> Yes I can view them. I notice they have the same content

4i. Delete the file "`rfc2186.txt`" and try to view the file "`rfc_2186.txt`" using the more command. Were you still able to view the file? Why or why not?

> Yes we can still view the file. This is because deleting a file does not exactly wipe the file out of the system. Deleting just removes our pointer to access that file. In this case, we deleted our original pointer but we still have rfc_2186.txt. So, we are still able to access and view the file. This is how hard links work.

4j. Delete the file "`rfc_2186.txt`" and copy the file "`rfc2186.txt`" using the command "`cp /home/files/rfc2186.txt rfc2186.txt`". Verify if the file was copied using the "`ls -l`" command.

4k. At the command prompt, type the command "`ln -s rfc2186.txt rfc_2186.txt`". This command creates a soft link (symbolic link) for the file "`rfc2186.txt`" text file. Verify if the soft link was created using the command "`ls -l`". Was the soft link created? How would you know if it were a soft link based on the output?

> Yes the soft link is created. We can tell it is a soft link because it has an arrow pointing to the original file

```
-rw-r--r-- 1 lbysyads18user20 students  18808 Jan 27 21:50 rfc2186.txt
lrwxrwxrwx 1 lbysyads18user20 students     11 Jan 27 21:50 rfc_2186.txt -> rfc2186.txt
lbysyads18user20@SYSSVR1:~$
```

4l. View the contents file "`rfc2186.txt`" by typing the command "`more rfc2186.txt`", to exit type "`q`". Also view the contents of the file "`rfc_2186.txt`" using the command "`more rfc_2186.txt`", to exit type "q". Were you able to view both files?

> Yes I can view both files

4m. Delete the file "`rfc2186.txt`" and try to view the file "`rfc_2186.txt`" using the more command. Were you still able to view the file? Why or why not?

> No, we can no longer view the file. This is because a soft link is simply a new file entry that points to the original file. Hence, one cannot exist without the other.

Delete the soft link created for the file "`rfc2186.txt`".

Step 5 At the command prompt, type the command "`ln -s /home/dir1 test_dir`".

5a. Was the soft link for the directory created?

> Yes the soft link was created

5b. How do you know if it is a soft link from the output of the "`ls -l`" command?

> I can see that it is a soft link because it has an arrow pointing to the original directory

```
rwxr-xr-x 2 lbysyads18user20 students   4096 Jan 20 16:25 dir2
rw-r--r-- 1 lbysyads18user20 students 131676 Jan 27 20:38 rfc1034.pdf
rwxrwxrwx 1 lbysyads18user20 students     10 Jan 27 21:55 test_dir -> /home/dir1
```

Step 6 On the command prompt type the command "`ls /home/dir1`" and then type the command "`ls test_dir`".

6a. Were the contents of the directories the same? Why or why not?

> Yes the contents of the file is the same. This is because a soft link does not create a copy. It is simply a new file entry that points to the same original file.

6b. Use the command "`rmdir test_dir`" to delete "`test_dir`". Were you successful? What was the message on the screen?

> No, the message said that it cannot be deleted because it is not a directory. This teaches us that soft links of directories are not entirely treated as directories. Commands like 'rmdir' would not work, we still use 'rm' if we would like to delete it.

5.3. File Ownership, File Permissions and Groups

Each file in the Unix system has an **owner and permission**. Each file is associated to an owner using its user ID. The username is just a way for us to remember more than just a bunch of numbers. A group is a logical grouping of users. Each group name in the Unix system has a group ID. A group may also have ownership of a file. Most of the time, the owner has access to the file and modifies the permission of a file.

File permission is a way to provide file security in the Unix system. When you enter the command "`ls -l`", notice the first column as shown in Fig 5.2. The first column in the output of the command shows the file permission of a file. The column is composed of 10 characters. The **first character** indicates whether the file is a **directory**, the next nine characters show the **permission** of the file that can be grouped into **three columns** where each column is composed of three characters. Each three-character column shows the file permission for the **owner**, **group**, and **world**. Each character in the column indicates a read, write or execute permission.

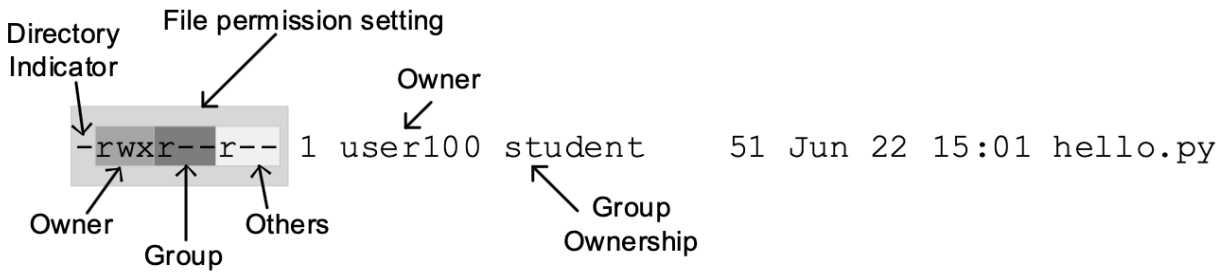

Fig. 5.2 Output of the "`ls -l`" command with file permission setting

Fig. 5.2 Output of the "`ls -l`" command with file permission setting

Looking at the owner setting at the example in Fig. 5.2, the "r" stands for **read access**, "w" stands for **write access** and "x" stands for **executable**. This means that the owner of the file has **read, write and execute** access. Note that the position for read, write and execute attributes are toggle bits. So if you see a "-" in the read position, this means that it has **no read access**. For group access, where in this case is for the student group, it only has read access. The "others" or rest of the world also has read only access. File permissions are also applicable to directories.

File permissions can be changed using the "`chmod`" command. The command uses integers to change the permission of a file. The command "`chmod`" is used in the form:

```
chmod vyz [file_name]
```

Where `vyz` is the **permission integer** to be put into the file. The `v` is actually a **binary representation** (seen as an octal number) for `rwx` permission. "r" is read, "w" is write and "x" is executable. If an "`r`" is a logic 1, then the file has read permission, if the "`r`" is logic 0 then the file has no read permission. The same rule applies to write (`w`) and execute (`x`) permission. The `y` and `z` are also binary representation for `rwx` permission. Example:

```
chmod 744 rfc2186.txt
```

This command sets the file permission of the text file "rfc2186.txt" as read, write and executable for owner, read for group and last, read for world (**`111`, `100`, and `100` in binary respectively**).

Step 7 At the command prompt, enter the command "`cp /home/files/rfc2186.txt rfc2186.txt`".

7a. What is the file permission of the owner for the file "`rfc2186.txt`"? Who is the owner? Which group does it belong to? What is the group's file permission?

> The owner is only allowed to read and write. The owner is stated in the 3rd column, which is lbysyads18user20. The group is stated in the 4th column, which is students. The group can only read the file.

7b. Enter the command "`chmod 444 rfc2186.txt`". Use a text editor and try to edit and save the file. Were you successful? Why or why not?

> No, it was not successful. This is because the permission we set 4 in binary is 100, this means only r is a logic "1". From this, we only have read permission.

7c. Enter the command "`chmod 644 rfc2186.txt`". Use a text editor and try to edit and save the file. Were you successful? Why or why not?

Yes we are successful. This is because 6 in binary is 110. So, read and write are logical "1" which means we can read and write.

7d. Enter the command "`cp /home/files/long_list long_list`". View the file using a text editor, `more` or `cat` command. What does the file contain? Does it a contain a command? What is the file permission for the owner?

```
lbysyads18user20@SYSSVR1:~$ more long_list
#!/bin/bash

echo "Hello, world!"
echo "The current date and time is: $(date)"
lbysyads18user20@SYSSVR1:~$ cat long_list
#!/bin/bash

echo "Hello, world!"
echo "The current date and time is: $(date)"
lbysyads18user20@SYSSVR1:~$ ls -l long_list
-rw-r--r-- 1 lbysyads18user20 students 79 Jan 27 22:33 long_list
lbysyads18user20@SYSSVR1:~$
```

The file contains a small program that prints "Hello, world!" and the current date and time. The owner only has read and write permission.

7e. Enter the command "`./long_list`" at the prompt. What happened? What was the message?

It was unsuccessful, the message stated permission denied. This command is to execute a program, but we only have read and write permission. Hence, we could not execute the program.

7f. Enter the command "`chmod 766 long_list`" at the prompt. What is the file permission for the owner after the command was executed?

The permission now allows the owner to read, write, and execute.

7g. Enter the command "`./long_list`" at the prompt. What happened? What is the command similar to?

The shell script was executed successfully. The result is,

Hello, world!
The current date and time is: Tue Jan 27 22:38:19 PST 2026

This command is similar to 'bash', which also executes shell script too.s

Step 8 At the prompt, enter the command "`ls -ld /home/student_files`".

8a. Who is the owner of the file? What is the file permission of the owner?

The owner is root. They have permission to read, write, and execute.

```
lbysyads18user20@SYSSVR1:~$ ls -ld /home/student_files
drwxrwxr-x 2 root students 4096 Jan 30 13:43 /home/student_files
lbysyads18user20@SYSSVR1:~$
```

8b. Which group does the directory belong to? What is the group permission? What is the world permission?

This directory belongs to the 'students' group. The group only has permission to read, write and execute. Whereas, the world only has permission to read and execute.

8c. At the prompt, enter the command "`groups`". To which group do you belong?

I belong to students group

8d. At the prompt, enter the command "`cd /home/student_files`". In the directory, enter the command "`touch <username>`". Were you successful? Why or why not?

> No, I was denied permission. The touch command needs write permission. I am part of the 'student' group which falls under 'other' for this directory. So, I am only allowed to read and execute permission.

8e. At the prompt, enter the command "`cd /home`" and also enter the command "`ls -ld faculty_files`". Who is the owner of the file? What is the file permission of the owner?

```
lbysyads18user20@SYSSVR1:/home$ ls -ld faculty_files
drwxrwxr-x 2 root faculty 4096 Jan 28 09:56 faculty_files
lbysyads18user20@SYSSVR1:/home$
```

The owner is root. The owner is allowed permission to read, write, and execute.

8f. Which group does the directory belong to? What is the group permission? What is the world permission?

> The directory belongs to the faculty group. The group can read, write, and execute. The world can only read and execute

8g. At the prompt, enter the command "`cd /home/faculty_files`". In the directory, enter the command "`touch <username>`". Were you successful? Why or why not?

> Most definitely not. This is due to the fact that the permission bestowed upon me is only read and execute as I am part of the students group.

5.4. Conclusion/Reflection

Observe the following file contents and command outputs for `usage.txt`:

```
  GNU nano 2.9.3 usage.txt

Hello World!

^G Get Help  ^O Write Out ^W Where Is
^X Exit      ^R Read File ^\ Replace
```

```
admin@sysadsvr:~$ ls -l usage.txt
-rw-r--r--   1 admin   faculty  13   Nov   5 2018   usage.txt
```

```
admin@sysadsvr:~$ du -h usage.txt
4.0K    usage.txt
```

Why does the "`ls -l`" command reports a file size of 13 bytes while the "`du -h`" command reports a 4 kilobytes disk usage for the file called "`usage.txt`"? You may use references to answer this question.

I utilized ChatGPT with the following prompt "With ls -l , what unit is used for the size of the file?" and "So, why for the same file using ls -l indicates a 13 bytes file size. But with du -h it indicates a 4.0K size"

I learned that ls -l states the actual file size while du -h states the minimum file block allocated to it. We can confirm these two values with the "stat" command, I tested with rfc2186.txt:

```
lbysyads18user20@SYSSVR1:~$ stat rfc2186.txt
  File: rfc2186.txt
  Size: 18817           Blocks: 40         IO Block: 4096   regular file
Device: fc36h/64566d    Inode: 1050951     Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1324/lbysyads18user20)   Gid: ( 1002/students)
Access: 2026-01-27 22:32:33.424356535 +0800
Modify: 2026-01-27 22:32:27.311880752 +0800
Change: 2026-01-27 22:32:27.311880752 +0800
 Birth: 2026-01-27 22:25:30.888469969 +0800
lbysyads18user20@SYSSVR1:~$ ls -l rfc2186.txt
-rw-r--r-- 1 lbysyads18user20 students 18817 Jan 27 22:32 rfc2186.txt
lbysyads18user20@SYSSVR1:~$ du -h rfc2186.txt
20K     rfc2186.txt
lbysyads18user20@SYSSVR1:~$
```

Here I can see that 40 blocks is used. So, 40 * 512 bytes = 20480 bytes = 20K. While the value 18817 is the actual file size in bytes.

How do you delete symbolic links for directories? Why does the "`rmdir`" command not work as shown in 6b?

We can treat it like a file and use the 'rm' command. This will remove multiple symbolic links at once. We can also use the 'unlink' command, this removes a symbolic link one at a time. The 'rmdir' command does not work because it is designed to delete an actual, empty directory. Not a link to one. Hence, the command does not work.

What will happen if the owner removes the execute permissions of a directory for the group or the world? Write your answer from the perspective of the other users (not the owner) from the groups/world.

If we do not have execute permissions, it will be as if we are locked out of the directory. The execute permission is what allows us to search and traverse through the directory. Without it, we cannot do basic commands like cd, ls, or cp. Our read and write permissions, though allowed, cannot dominate the execute permission.